

End goal of data: Use YelpAPI and a csv about population by city in California to find the correlation between California restaurant ratings and city population. I will analyze the data by finding averages of restaurant ratings & number of reviews by city; I will make a correlation cell calculating the percent correlation between the two.

```
import pandas as pd
from datetime import datetime
import seaborn as sns
import numpy as np
!pip install yelpapi

Requirement already satisfied: yelpapi in /usr/local/lib/python3.10/dist-packages (2.5.1)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from yelpapi) (2.31.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->yelpapi) (3.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->yelpapi) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->yelpapi) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->yelpapi) (2024.

# Import necessary libraries
from yelpapi import YelpAPI
from pandas import json_normalize

# Define my Yelp API key
api_key = '5Ku7XA4CNM1CC0z-tc8gJRqm6DnTD07oZQAiM0CyyhBYCPg3RY9BiL7C-zure5Tqtw0mPjRPnsqJpL_fiDXJBMftMteW6521ELpbUuXIldseMYgQ5pegh7'

# Create a YelpAPI instance with your API key
yelp_api = YelpAPI(api_key)

# Import the requests library
import requests

# Set up the headers for the API request
headers = {
    'Authorization': f'Bearer {api_key}',
}

#connection information
!pip install mysql-connector-python
import mysql.connector
import pandas as pd

#SQL connection information
mysql_address = 'ista322server.chm4wokly2d0h.us-east-2.rds.amazonaws.com'
mysql_username='avamizrahi'
mysql_password='Avamizrahi2'
mysql_database = 'ista322dbs'

def run_query(query_string):

    # get connection and cursor
    conn, cur = get_conn_cur()

    # executing string
    cur.execute(query_string)

    # fetch query data
    my_data = cur.fetchall()
    result_df = pd.DataFrame(my_data, columns=cur.column_names)

    #close
    cur.close()
    conn.close()

    return result_df

def get_conn_cur():
    # Connect to the MySQL database using the provided credentials
    cnx = mysql.connector.connect(user=mysql_username, password=mysql_password,
                                  host=mysql_address,
                                  database=mysql_database, port='3306');
    return (cnx, cnx.cursor())
```

Requirement already satisfied: mysql-connector-python in /usr/local/lib/python3.10/dist-packages (8.3.0)

#Get a connection and cursor using the get_conn_cur() function

```
conn, cur = get_conn_cur()
conn
```

```
<mysql.connector.connection_cext.CMySQLConnection at 0x7b23a42978b0>
```

#Define a function to insert data into the 'CA_population_data' table

```
def insert_data(df):
    conn, cur = get_conn_cur()

    # SQL statement for inserting data
    insert_query = """
    INSERT INTO CA_population_data (city, most_current_pop_year, population)
    VALUES (%s, %s, %s)
    """
```

```
try:
```

```
    #Iterate over each row in the DataFrame and insert the data into the table
    # name=None generates tuples
    for row in df.itertuples(index=False, name=None):
        cur.execute(insert_query, row)
```

```
    # Commit the transaction to save the changes
    conn.commit()
    print("Data inserted successfully.")
```

```
# Print an error message if an exception occurs
except Exception as e:
    print(f"Error inserting data: {e}")
```

```
cur.close()
conn.close()
```

Use the Yelp API to search for sandwich shops in California

```
sandwich_shops = yelp_api.search_query(term = 'sandwich', location = 'California')
sandwich_shops
```

Import the drive module from the google.colab library

```
from google.colab import drive
```

Mount the Google Drive to access files

```
drive.mount('/content/drive')
```

Define the file path to the CSV file in my Google Drive

```
file_path = '/content/drive/My Drive/California_DemographicsByCity_sample 2.csv'
```

Open the file in read mode with UTF-8 encoding

```
with open(file_path, 'r', encoding='utf-8') as file:
```

```
    # Read the first 5 lines of the file and print them
    for i in range(5):
        print(repr(file.readline()))
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
'city,state,county,most_current_pop_year,population\n'
'city,,county,most_current_pop_year,population\n'
'California,,2022,"39,356,104"\n'
'Acalanes Ridge CDP,,Contra Costa,2022,961\n'
'Acampo CDP,,San Joaquin,2022,255\n'
```

Read the CSV file into a pandas DataFrame to allowing me to analyze & manipulate it using pandas

```
CA_population_data = pd.read_csv(file_path)
```

```
# Remove the first row from the DataFrame
CA_population_data = CA_population_data.drop(CA_population_data.index[0])

# Remove rows 1611 to 1616 from the DataFrame
CA_population_data = CA_population_data.drop(index=range(1611, 1617), errors='ignore')

# Remove the 'state' and 'county' columns
CA_population_data = CA_population_data.drop(columns=['state', 'county'])

# Reset the index after dropping rows
CA_population_data.reset_index(drop=True, inplace=True)

# Remove commas from 'most_current_pop_year' and 'population' columns
CA_population_data['most_current_pop_year'] = CA_population_data['most_current_pop_year'].str.replace(',', '')
CA_population_data['population'] = CA_population_data['population'].str.replace(',', '')

# Convert 'most_current_pop_year' and 'population' columns to int
CA_population_data['most_current_pop_year'] = CA_population_data['most_current_pop_year'].astype(int)
CA_population_data['population'] = CA_population_data['population'].astype(int)

# Display the first few rows of the DataFrame
print(CA_population_data.head())

# Insert data into the SQL table
insert_data(CA_population_data)
```

	city	most_current_pop_year	population
0	California	2022	39356104
1	Acalanes Ridge CDP	2022	961
2	Acampo CDP	2022	255
3	Acton CDP	2022	7286
4	Adelanto city	2022	37960

Data inserted successfully.

```
# Add a new column 'state' to the CA_population_data DataFrame with the value 'CA' for every row
CA_population_data['state'] = 'CA'
CA_population_data
```

	city	most_current_pop_year	population	state
0	California	2022	39356104	CA
1	Acalanes Ridge CDP	2022	961	CA
2	Acampo CDP	2022	255	CA
3	Acton CDP	2022	7286	CA
4	Adelanto city	2022	37960	CA
...	...	...	...	...
1605	Yosemite West CDP	2022	29	CA
1606	Yountville city	2022	3397	CA
1607	Yreka city	2022	7833	CA
1608	Yuba City city	2022	69535	CA
1609	Yucaipa city	2022	54428	CA

1610 rows x 4 columns

Next steps: [View recommended plots](#)

```
# Flatten sandwich_shops DataFrame to JSON
sandwich_shops_df = pd.json_normalize(sandwich_shops['businesses'])

# Print the JSON data
print(sandwich_shops_df)
```

```

# Initialize an empty list to store individual shop reviews
sandwich_shops_reviews_df = []

# Iterate over each row in sandwich_shops_df
for i in range(len(sandwich_shops_df)):

    # Get the location ID of the shop
    location_id = sandwich_shops_df.loc[i, 'id']

    # Query Yelp API for reviews of the shop
    reviews = yelp_api.reviews_query(location_id)

    # Normalize the reviews into a DataFrame and add the location ID as a column
    reviews_df = pd.json_normalize(reviews, 'reviews')
    reviews_df['location_id'] = location_id

    # Append the DataFrame to the list
    sandwich_shops_reviews_df.append(reviews_df)

# Concatenate all reviews into a single DataFrame
sandwich_shops_reviews_df = pd.concat(sandwich_shops_reviews_df)

# Initialize an empty list to store individual shop reviews
sandwich_shops_reviews_list = []

# Iterate over each row in sandwich_shops_df
for i in range(len(sandwich_shops_df)):
    # Get the location ID, state, and city
    location_id = sandwich_shops_df.loc[i, 'id']
    state = sandwich_shops_df.loc[i, 'location.state']
    city = sandwich_shops_df.loc[i, 'location.city']

    # Query Yelp API for reviews
    reviews = yelp_api.reviews_query(location_id)

    # Normalize reviews and add 'state' and 'city' columns
    reviews_df = pd.json_normalize(reviews, 'reviews')
    reviews_df['state'] = state
    reviews_df['city'] = city

    # Append to the list
    sandwich_shops_reviews_list.append(reviews_df)

# Concatenate all reviews into a single DataFrame
sandwich_shops_reviews_df = pd.concat(sandwich_shops_reviews_list)
sandwich_shops_reviews_df.head()

# Initialize an empty list to store individual shop reviews
sandwich_shops_reviews_list = []

# Iterate over each row in sandwich_shops_df
for i in range(len(sandwich_shops_df)):
    # Get the location ID, state, and city
    location_id = sandwich_shops_df.loc[i, 'id']
    state = sandwich_shops_df.loc[i, 'location.state']
    city = sandwich_shops_df.loc[i, 'location.city']

    # Query Yelp API for reviews
    reviews = yelp_api.reviews_query(location_id)

    # Normalize reviews and add 'state' and 'city' columns
    reviews_df = pd.json_normalize(reviews, 'reviews')
    reviews_df['state'] = state
    reviews_df['city'] = city

    # Append to the list
    sandwich_shops_reviews_list.append(reviews_df)

# Concatenate all reviews into a single DataFrame
sandwich_shops_reviews_df = pd.concat(sandwich_shops_reviews_list)

# Select specific columns from the DataFrame
sandwich_shops_reviews_df = sandwich_shops_reviews_df[['id', 'text', 'rating', 'state', 'city']]

```

```
# Merge both data sets on the 'state' and 'city' columns
merged_data = CA_population_data.merge(sandwich_shops_reviews_df, on=['state'], how='inner')

# Reset the index
merged_data.reset_index(drop=True, inplace=True)

merged_data
```

```
# Extract 'population' and 'rating' columns
extracted_data = merged_data[['population', 'city_y', 'rating']].copy()
extracted_data
```

	population	city_y	rating	
0	39356104	Shaver Lake	5	
1	39356104	Shaver Lake	5	
2	39356104	Shaver Lake	5	
3	39356104	Oakhurst	5	
4	39356104	Oakhurst	5	
...	...	...	...	
96595	54428	El Portal	4	
96596	54428	El Portal	4	
96597	54428	Oakhurst	5	
96598	54428	Oakhurst	4	
96599	54428	Oakhurst	5	

96600 rows x 3 columns

Next steps:

 [View recommended plots](#)

```
# Calculate the correlation between the 'population' and 'rating' columns in the 'extracted_data' DataFrame
correlation = extracted_data['population'].corr(extracted_data['rating'])

# 'correlation' now contains the correlation coefficient between the 'population' and 'rating' columns
correlation

1.5561882801065136e-18
```

This number shows a small positive correlation between restaurant ratings in California and popularity of California cities. The correlation is very small and positive, which means that sometimes the more populated a city is, their restaurants may have higher ratings. Vice versa, the less populated a California city is, restaurants may be rated lower